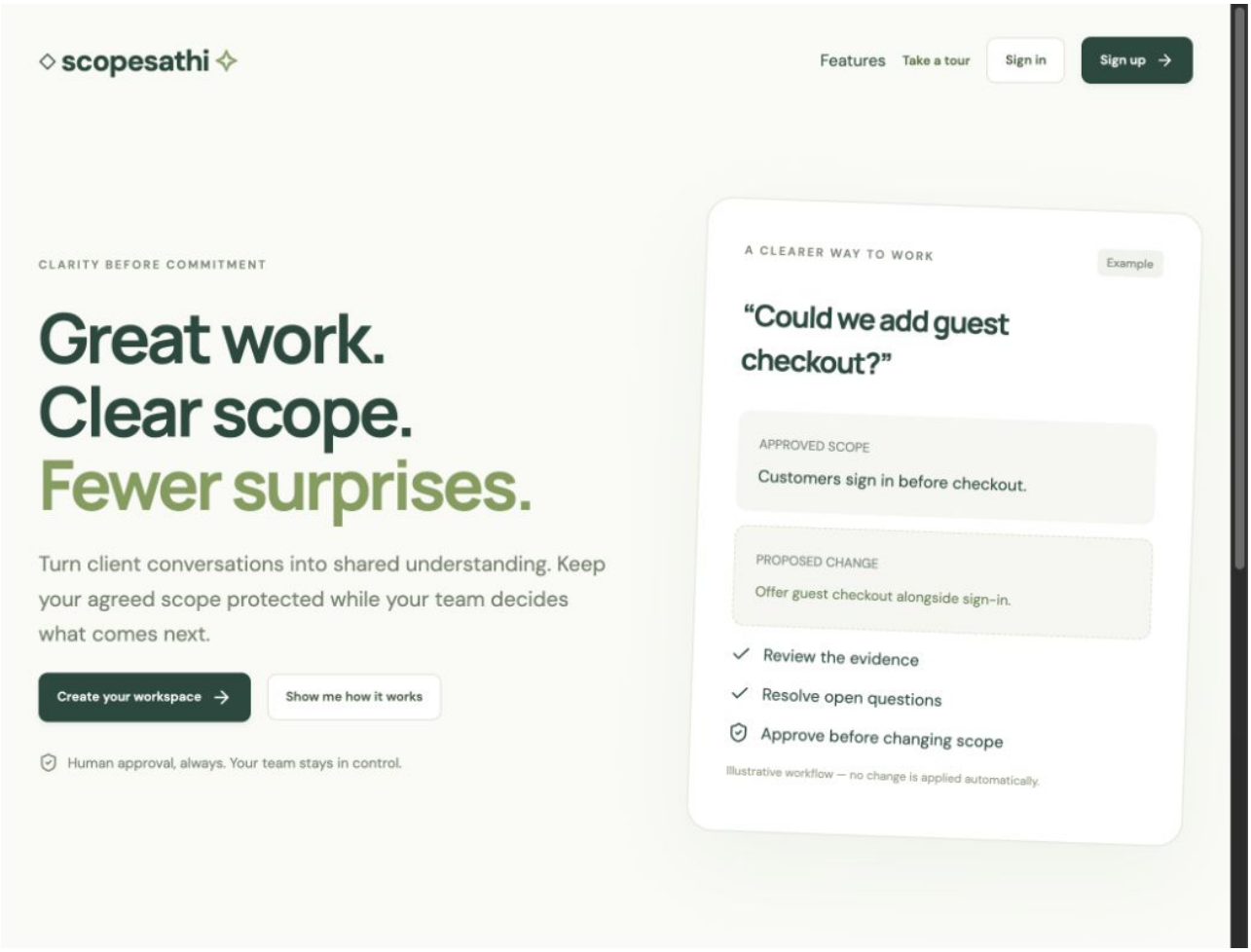
ScopeSathi

Clarity before commitment.

A requirements and scope-change workspace for freelancers and small software teams. Prepared for the Future of Work & Enterprise Productivity theme.



Prototype status: core manual workflows and Firestore persistence tested. Live Gemini analysis and public deployment remain pending. This report describes the current implementation honestly; it is not a claim of a completed production release.

Author: Shaurya Verma
Source: github.com/shauryaverma03/Scope-Sathi

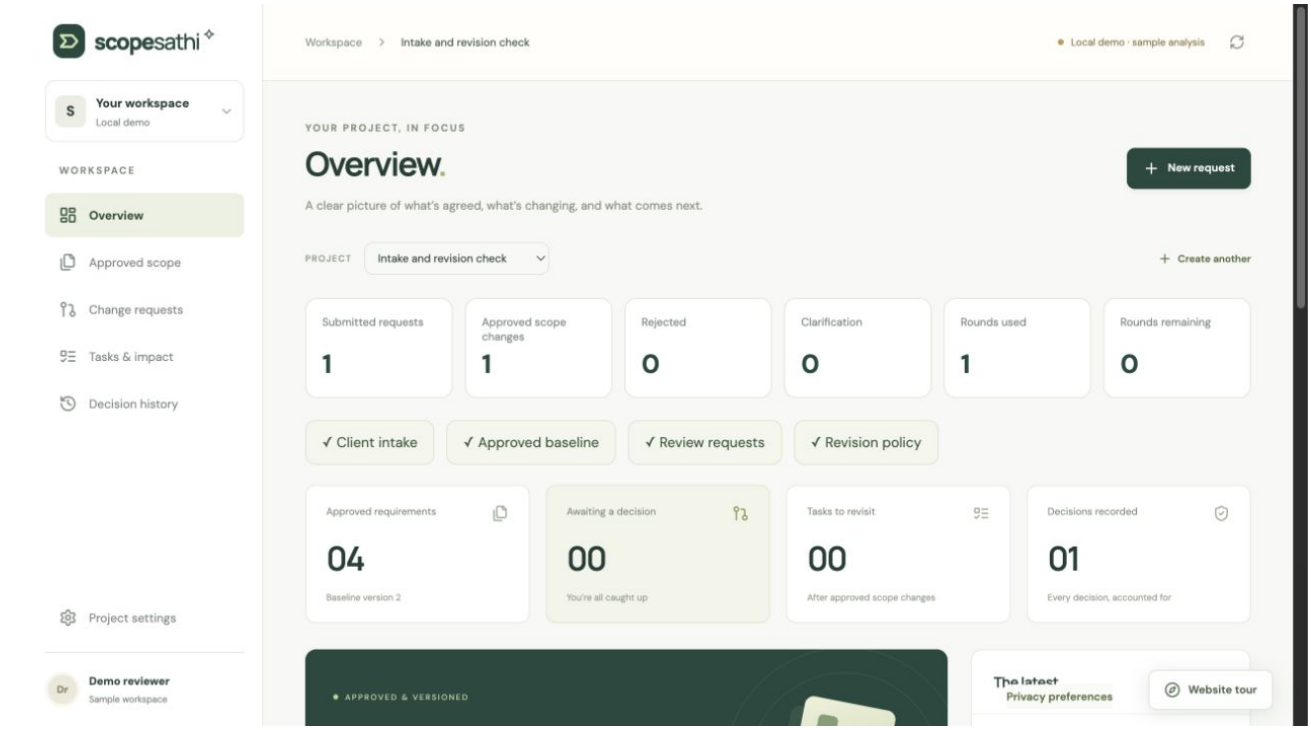
01 / The problem and the solution

A client agrees to a sign-in-only checkout, then asks for guest checkout near delivery. Teams need to know which agreement changes, who approved it and whether a revision allowance is consumed.

Stage	What the user does and what the app preserves
Capture	Create the project and record goal, users, features, deliverables, constraints, acceptance criteria and exclusions. Paste a brief or upload TXT/selectable-text PDF.
Review baseline	Review extracted requirements, edit wording and preserve exact source passages. A reviewer approves version 1.
Compare request	Submit the client message. Compare requirement IDs, old wording, request quotes, proposed wording and linked tasks. Multiple findings can describe one request.
Decide	Approve, reject or ask for clarification with a reason. Open questions and stale revisions block approval.
Follow through	Approval preserves previous versions and flags affected tasks. History records reviewer and reason. Exports provide scope/history JSON and task CSV.

AI proposes; people decide. Without a model connection, extraction is deterministic text splitting and request review is manual. The prepared checkout example is explicitly a demo, not live AI.

02 / Workspace and collaboration



Actual application capture with demonstration data. Counts distinguish requests, approved scope changes and revision rounds.

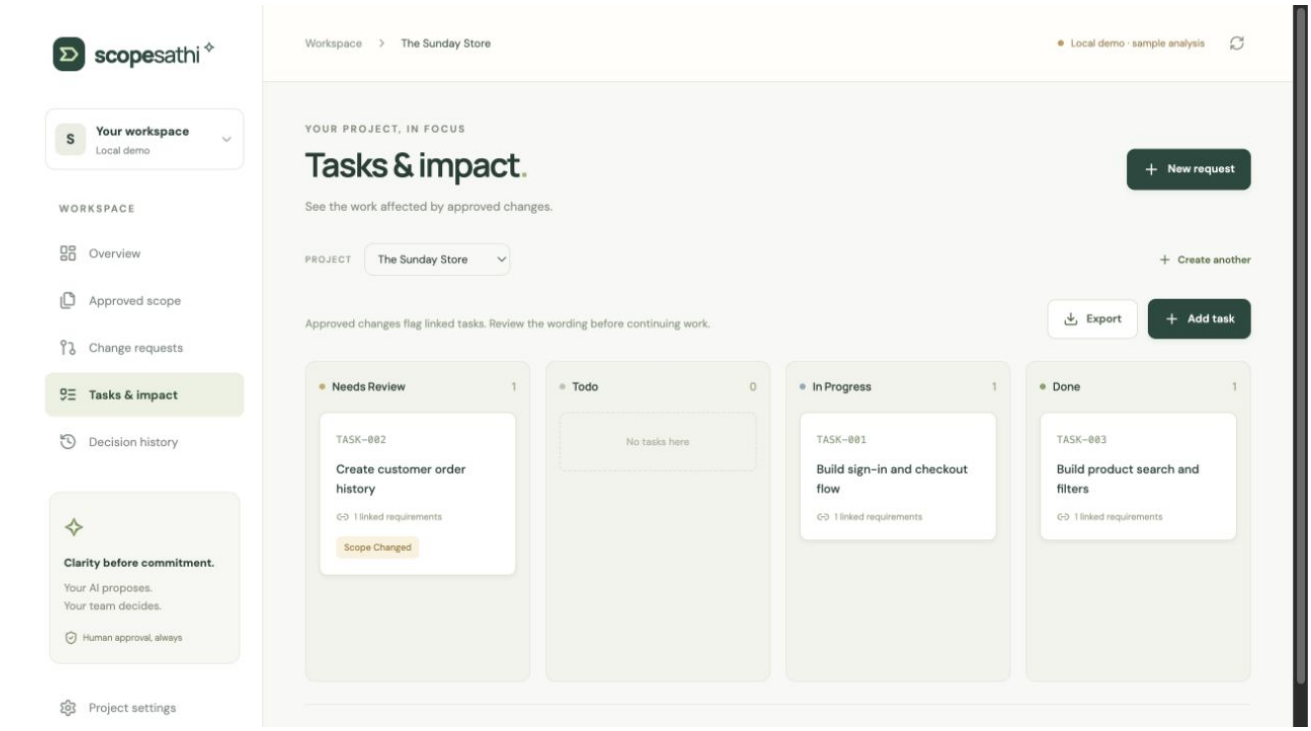
Section	Purpose
Overview	Project summary, setup checklist, pending decisions, affected tasks and allowance.
Approved scope	Intake, upload, reviewed requirements, baseline approval and version comparison.
Change requests	Request list, search/status filters, finding editor and reviewer decisions.
Tasks and history	Linked task board, needs-review work, chronological reasons and exports.
Settings and onboarding	Account/connection status, owner-managed access and revision policy. Google sign-in, tour, session preferences and reduced-motion loading.

Invitations grant access only after acceptance by the matching verified Google account. Owners can revoke pending invitations; invitations expire after seven days. Unknown recipients require owner confirmation. Welcome and invitation templates have HTML and plain-text versions.

03 / Revision policy and task impact

The owner defines the included number of revision rounds and what a round means. Related requests can share a round. Each approved new-scope round is counted once, even if it includes multiple requests.

Corrections needed to satisfy the original agreement, rejections, clarification and no-change findings do not consume allowance. Approval beyond the allowance requires a written separate agreement. ScopeSathi never automatically charges the client.



Actual demonstration task board after approval; affected work moves to Needs Review.

Example: two approved changes in the same launch-review round consume one round. A second round beyond a one-round allowance requires a separate agreement. The affected task remains subject to a human task review.

Roles: owners control membership, deletion and policy; reviewers approve baselines and decisions; editors submit requests/manage tasks; viewers read/export. Backend checks enforce access independently of the UI.

04 / Architecture and technology

Layer	Implementation and reason
Browser	React + TypeScript + Vite: forms, comparisons, task board and onboarding. Firebase Authentication supplies Google identity.
API boundary	Express + Firebase Admin: verify identity, rate-limit requests, validate input with Zod and enforce project roles.
Domain rules	Exact quotes, requirement/task references, unresolved questions, baseline versions and revision allowances gate decisions.
Persistence	Firestore stores project aggregates, users, notifications and mail outbox. Transactions and revision checks prevent lost updates.
Analysis	Google Gen AI SDK supports Developer API in local firebase mode and Vertex AI in google mode. Not connected in the tested installation.
Email	Nodemailer submits branded messages through Gmail SMTP. The persisted queue is processed by a backend timer.
Deployment	Cloud Run can serve API + bundled frontend; Firebase Hosting is optional. Configuration exists, but public deployment is pending.

Approval request path

Browser + ID token / API authentication / role and evidence checks / Firestore transaction / updated baseline/history/tasks / confirmed response

Model calls occur outside transactions. The write later checks the original revision. A stale client gets 409 rather than overwriting another edit. Direct browser Firestore access is denied by the supplied rules; server Admin access still requires application checks and IAM.

Original PDF bytes are discarded after extraction. The product retains text and reviewed history; no Cloud Storage upload bucket is required.

05 / Test results and honest limits

Verification date: 14 October 2026. Automated tests ran against isolated fixtures. The live database test used a temporary project and cleaned it up.

Check	Result / evidence boundary
Automated suite	39/39 tests passed. Includes baseline approval, quotes, ambiguous findings, multi-change review, roles, concurrency, revision allowance and invitation validation.
Production build	TypeScript and Vite production build passed.
Live Firestore	Create, approve baseline, submit/approve request, task impact/update, outsider denial and fresh-process persistence passed.
Runtime configuration	Firebase mode, Firestore read connection verified, email configured, AI mode manual.
UI evidence	Real screenshots from prior local walkthroughs. A complete fresh browser regression and a new Google popup login were not performed in this run.
Email delivery	Template and invitation logic tests passed. Prior styled test mail was received; this run did not send mail or verify a new-user inbox.
Live AI / hosting	Not verified: Gemini credentials/model access and public Cloud Run/Hosting deployment remain pending.

This is not a load test or a complete security audit. Passing local tests does not establish real model quality, email inbox delivery or production availability.

06 / Work to complete before submission

Priority	Action and completion check
1 - Live analysis	Configure a private Gemini key and model for local firebase mode, or Vertex AI/ADC for google mode. Test actual requirement extraction and multi-change matching with exact quotes; record results.
2 - Public deployment	Resolve Google billing/credits and organizer requirements. Deploy to the eligible Google platform, authorize the hostname and verify sign-in plus a saved project reload.
3 - Email workflow	Verify welcome once on first signup, invitation delivery to a permitted recipient, acceptance and role access. Test unknown addresses and invalid format. SMTP acceptance does not prove delivery.
4 - Reliable email worker	Replace reliance on a polling timer for scale-to-zero hosting, or deliberately provision always-running processing. The current timer can pause when hosting sleeps.
5 - Final UI regression	Verify desktop/mobile, all sections, exports, tour, profile, loading/error paths and a real two-user collaboration session.
6 - Scale and recovery	Add distributed rate budgets if using replicas, pagination/subcollections beyond the 700 KB project cap, backup/restore policy and monitoring.

Already implemented protections: per-user and analysis limits, ingress limits, invitation/session budgets, request IDs, JSON/upload limits, security headers, role checks, transaction revisions and HTTP shutdown/timeouts.

Not implemented: OCR, automatic invoices, Jira sync, durable hosted mail scheduling, shared/global rate counters, semantic model accuracy evaluation or general idempotency keys.

Repository documentation: docs/README.md links architecture, API, data model, configuration, security, email and validation guides. Screenshots show demo data and are not cloud-connectivity proof. No secrets are included in this report.

07 / Demo script and acceptance checks

Upload the accompanying Sunday Store client-brief PDF to a new test project. It contains only initial requirements; the change request below is intentionally separate.

Step	Expected result
1. Create project	Name it Sunday Store Demo. Set an included allowance of one round.
2. Upload brief	Approved scope extracts selectable text. Without Gemini, review and consolidate text sections manually; do not call that semantic AI analysis.
3. Approve baseline	Verify sign-in-only checkout, search/filtering, order history and exports exclusion. Approve once.
4. Link a task	Create Build checkout linked to the actual checkout requirement ID. IDs should be read from your reviewed baseline, not assumed.
5. Submit change	Use the client message below. Manually create two findings when AI is absent.
6. Resolve and approve	Confirm wording and task links, clear questions, record reason and approve. Check preserved version and task Needs Review.
7. Check allowance	Both changes in one approved round use one round. A new round beyond it requires a separate agreement.
8. Reopen	Reload and confirm saved data. Export scope/history/tasks as needed.

Client change request: Please let customers check out as guests as well as sign in. Guest customers must receive an email confirmation with an order-tracking link that expires after 24 hours. Also add a CSV export of customer order history.

Expected interpretation: checkout is a modification; CSV export adds scope that was explicitly excluded. Human review must resolve the exclusion consistently. In manual mode, adjust the relevant exclusion wording or remove it as part of the reviewed proposal; do not leave contradictory requirements.

Ambiguity test: submit “Make checkout easier.” Keep it as clarification until the client explains what should change. Rejection/clarification must not consume another round.

Submitting this PDF alone does not satisfy a requirement for a live deployed prototype. Update the status and demo evidence after completing the pending integration work.